

AGRIBID SHUDH · FRONTEND LEARNING GUIDE

How the Admin Frontend Fits Together

A beginner-friendly map of the React and Next.js code built for the M00 foundation and M01 authentication screens.

Current state: the visual frontend and shared foundations are built. Authentication is not connected to a live backend yet.

Next.js App Router

React + TypeScript

Tailwind CSS

shadcn/ui

TanStack Query

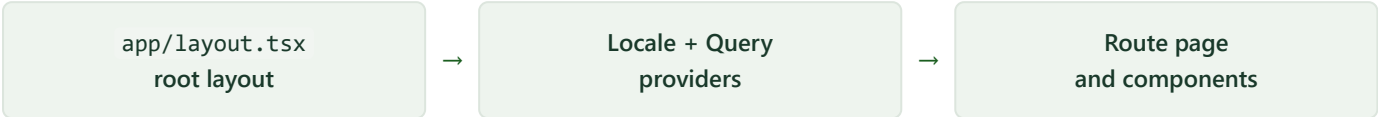
Motion installed

Read this alongside the source files. It explains the purpose of each layer, not every Tailwind utility class.

01 · BIG PICTURE

Routes and component flow

In the App Router, folders under `app/` become URL paths. A page exports the UI for its route; layouts and providers wrap pages.



URL	File	Purpose
/	app/page.tsx	Dashboard route, rendered inside AdminShell.
/admin/login	app/admin/login/page.tsx	Email/password screen and optional expired-session notice.
/admin/2fa	app/admin/2fa/page.tsx	Standalone TOTP entry screen.

What the root layout does

- `app/layout.tsx` sets metadata, loads Geist fonts, imports global CSS, and wraps all routes in `LocaleProvider` and `QueryProvider`.
- `children` is the page selected by the current URL. The root layout does not perform login.
- The layout is a Server Component. Client providers can be rendered inside it to enable browser-side state below that boundary.

Server vs Client: a page is a Server Component by default. Add `"use client"` when that component needs state, event handlers, effects, or browser APIs. Login and TOTP forms use it because they track typed input.

Reusable layout pieces

- `components/layout/AdminShell.tsx` gives the dashboard its header, navigation, and content area.
- `components/auth/AdminAuthLayout.tsx` shares the branded responsive panel between login and TOTP.
- Both layouts receive children through a prop. This avoids duplicating shared markup.

02 · STATE AND LANGUAGE

How the screens respond

Controlled login fields

- 1 `useState("")` creates a value and setter for email and password.
- 2 `value={email}` displays the current React state in the input.
- 3 `onChange` calls `setEmail(event.target.value)` as the user types. This makes the input controlled by React.
- 4 `handleSubmit` calls `event.preventDefault()` to stop browser navigation. It intentionally does not authenticate yet.

TOTP form

The code field uses the same controlled-input pattern. `inputMode="numeric"` suggests a number keypad and `autoComplete="one-time-code"` helps browsers recognize an OTP field. No code length is enforced because the specification does not set it.

Session-expired notice

`components/auth/SessionExpiryNotice.tsx` reads the URL query string. The login page wraps it in `<Suspense>`, as required for Next.js prerendering with `useSearchParams`. Visit `/admin/login?reason=idle_timeout` to see the message.

Locale provider

- `lib/messages.ts` contains English (`en`), Hindi (`hi`), and Marathi (`mr`) text.
- `LocaleProvider` stores the selected language in React state and exposes `t(key)` through context.
- `useLocale()` is a custom hook that lets child components read the language and translations.
- The choice currently resets on a full reload. Backend `preferredLanguage` can become the source of truth after `/me` is integrated.

Beginner model: Props pass values down one component branch. Context shares a value across many descendants. State stores a value that can change while the page is open.

The visible UI is static with respect to authentication; no fake success response is shown.

03 · API FOUNDATION

How real API integration will fit



This is the intended separation: the UI collects input, a TanStack Query mutation manages an action, a service owns the HTTP call, and the backend verifies credentials. That flow is **not wired yet**.

TanStack Query provider

`components/providers/QueryProvider.tsx` creates one `QueryClient` for the app and makes it available to descendant components. No query or mutation is currently registered.

Shared request helper

`lib/api-client.ts` exposes `apiRequest<TResponse>(path, options)`. It reads `NEXT_PUBLIC_API_BASE_URL`, JSON-encodes a supplied body, sets `Accept-Language`, optionally adds a Bearer token, disables caching by default, and turns the documented error envelope into an `ApiError`.

Safety: the helper does not store tokens. No API base URL is configured, no request is made, and no real credentials are sent by the current forms.

What the M01 contract still needs to tell us

- Exact login and 2FA request/response JSON, including how the temporary 2FA challenge is represented.
- Where browser sessions and refresh tokens live, and how refresh/revocation works.
- How `/me` supplies role/permission data from M02 and how idle timeout revokes the server session.

Queries usually read server state. Mutations submit actions such as login. Login must be a mutation once the real contract is available.

04 · READING AND TESTING

A good source-reading order

- 1 `app/layout.tsx` — root wrapper and providers.
- 2 `components/providers/LocaleProvider.tsx` — context, state, and custom hook.
- 3 `app/admin/login/page.tsx` — controlled form, client boundary, and submit handler.
- 4 `components/auth/AdminAuthLayout.tsx` — reusable layout and props.
- 5 `app/admin/2fa/page.tsx` and `components/auth/SessionExpiryNotice.tsx` — second factor UI and query-based notice.
- 6 `components/dashboard/DashboardContent.tsx` — dashboard image, honest empty state, and documented distribution chain.
- 7 `lib/api-client.ts` and `components/providers/QueryProvider.tsx` — API foundation, not an auth implementation.

Run the checks

In PowerShell, use `npm.cmd` if the script policy blocks `npm.ps1` :

```
npm.cmd run lint
```

```
npm.cmd run build
```

```
npm.cmd run dev
```

Then check `http://localhost:3000/` , `/admin/login` , `/admin/2fa` , and `/admin/login?reason=idle_timeout` .

Current status

Built:

Admin shell, dashboard visual, localized static login/TOTP screens, idle-timeout message, Query provider, generic API helper.

Waiting for backend:

Real login/2FA, token lifecycle, protected routes, role-based navigation, live metrics, and actual session timeout/revocation.

The farmer image is a local Pexels photo at `public/farmer-field.jpg` . Motion is installed but intentionally unused until a transition improves a real interaction.